AACIP — Viva Presentation Script & Technical Defense Guide

Project: AI-Agentic Cyber Investigation Platform (AACIP)
Guide: Dr. Vanitha Soman
Presentation File: `docs/AACIP.pptx` (19 Slides)
Team Members:
1. **Pooja Mahale** (Slides 1 – 4)
2. **Pradnyesh Mhatre** (Slides 5 – 8)
3. **Angad Mistry** (Slides 9 – 14)
4. **Atharv Kumbhar** (Slides 15 – 19)

---

1. Domain Knowledge & Responsibility Matrix

Team Member	Domain Responsibility	Key Concepts to Defend	Core Implementation Files

Pooja Mahale	**Data Engineering & Forensic Pre-Processing**	• RFC 3164/5424 Syslog (`auth.log`) & Apache/Nginx formats
		• SHA-256 chain of custody & indicator defanging (`hxxp://`)
		• PCAP packet parsing & layer decoding
			`web_log_parser.py`
			`auth_log_parser.py`
			`pcap_parser.py`

Pradnyesh Mhatre	**Testing, Validation & Model Evaluation**	• Traditional LLM failure modes in incident response
		• 244-test automated suite (Unit, Integration, Security)
		• 0.0% entity hallucination benchmarking & adversarial testing
			`test_target_types_pipeline.py`
			`test_cyber_reasoning.py`
			`reproduce_results.py`

Angad Mistry	**Multi-Agent Orchestration & Full-Stack Platform**	• Asynchronous triage state machine & multi-agent pipeline
		• LLM inference router (NVIDIA NIM, DeepSeek-R1, Nemotron)
		• Dynamic Evidence DAG generation & interactive SVG visualizer
			`triage.py`
			`investigation.py`
			`reasoning.py`
			`app.js`

Atharv Kumbhar	**Invariant Engine, MITRE STIX & Database Backend**	• Machine-checked Grounding Invariants (G0–G4, G7)
		• 4-tier Epistemic Taxonomy (FACT, INFERENCE, HYPOTHESIS, UNKNOWN)
		• MITRE ATT&CK STIX 2.1 taxonomy engine & SQLAlchemy ORM
			`mitre.py`
			`models.py`
			`risk.py`

2. Concise Slide-by-Slide Presentation Script

Speaker 1: Pooja Mahale (Slides 1 to 4)

- **Slide 1 (Title):**

  > *"Respected external examiner and guide Dr. Vanitha Soman: Good morning. We are presenting **AACIP — AI-Agentic Cyber Investigation Platform**. My team members are Pradnyesh Mhatre, Angad Mistry, Atharv Kumbhar, and myself, Pooja Mahale."*

- **Slide 2 (Table of Contents):**

  > *"Our presentation covers four phases: operational background and literature review by me; limitations, problem statement, and tech stack by Pradnyesh; system architecture and live results by Angad; and formal verification, evaluation, and future roadmap by Atharv."*

- **Slide 3 (Introduction & Key Features):**

  > *"Modern SOC analysts face severe alert fatigue across heterogeneous telemetry. Standard conversational LLMs fail because they treat forensic investigations as open text generation. AACIP introduces **Architecturally Enforced Grounding**, ensuring AI agents reason solely over deterministically extracted, verified evidence."*

- **Slide 4 (Literature Review):**

  > *"Our work builds on three foundations: the Ji et al. ACM survey on LLM hallucinations, NIST SP 800-86 standards for forensic chain-of-custody, and the MITRE ATT&CK taxonomy. We identified a core gap: no existing platform enforces programmatic invariants preventing hallucinated indicators during triage. I will now hand over to Pradnyesh."*

Speaker 2: Pradnyesh Mhatre (Slides 5 to 8)

- **Slide 5 (Limitations of Traditional LLMs in DFIR):**

  > *"Thank you, Pooja. Traditional LLMs present four fatal flaws in DFIR: entity hallucination of fake IPs or hashes, complete lack of provenance, uncalibrated confidence on ungrounded claims, and non-deterministic verdicts across runs."*

- **Slide 6 (Problem Statement):**

  > *"Our problem statement is to build an autonomous multi-agent platform that ingests multi-modal telemetry, extracts deterministic evidence with immutable provenance, maps adversary behavior to MITRE ATT&CK, and mathematically guarantees zero entity fabrications."*

- **Slide 7 (Proposed Solution & Core Axiom):**

  > *"Our solution follows one core axiom: **'Deterministic tools discover evidence; LLMs only reason over discovered evidence.'** We enforce formal invariants G0 through G4 and categorize all findings into four epistemic tiers: FACT, INFERENCE, HYPOTHESIS, and UNKNOWN."*

- **Slide 8 (Technologies Used):**

> *"We built AACIP using Python 3.12 with strict mypy typing, FastAPI and Pydantic v2 for high-throughput validation, SQLAlchemy 2.0 with Alembic, NVIDIA NIM inference for DeepSeek-R1 and Nemotron-70B, and Argon2 with PyJWT for authentication. I will now hand over to Angad."*

Speaker 3: Angad Mistry (Slides 9 to 14)

- **Slide 9 (System Architecture):**

> *"Thank you, Pradnyesh. Slide 9 presents our three-layer architecture: an Ingestion Layer supporting 8 telemetry modalities, a Deterministic Core for extraction and MITRE correlation, and an Agentic Presentation Layer managing LLM reasoning and real-time visualization."*

- **Slide 10 (Architecture Components & Working Flow):**

> *"When raw telemetry arrives, we compute an immediate SHA-256 digest for chain-of-custody. Deterministic parsers extract concrete indicators into an immutable Evidence Directed Acyclic Graph (DAG). The Triage Agent then prompts the reasoning LLM strictly using these evidence IDs, and the Invariant Validator verifies output before storage."*

- **Slides 11 & 12 (Results: Login & Investigation Dashboard):**

> *"Slide 11 shows the authenticated analyst portal. Slide 12 shows our live dashboard, where an analyst can submit files, logs, or network captures. In this SSH attack demo, AACIP created an isolated investigation session, assigned an immutable UUID, and executed automated triage."*

- **Slide 13 (Results: Dynamic Evidence Graph):**

> *"Slide 13 displays our interactive SVG Evidence Graph. The central red node indicates the primary threat, blue nodes represent extracted factual entities such as source IP and target user, and orange nodes reflect mapped MITRE ATT&CK techniques. Every edge captures verified provenance; clicking any node reveals the exact log line."*

- **Slide 14 (Results: Final Investigation Report):**

> *"Slide 14 shows the synthesized report: an executive verdict, calibrated risk score (0–100), MITRE TTPs, and containment recommendations. Every statement includes an Evidence Pointer Badge allowing analysts to audit claims back to raw database rows. I now invite Atharv to present our validation and conclusion."*

Speaker 4: Atharv Kumbhar (Slides 15 to 19)

- **Slide 15 (Evidence Validation & Invariant Matrix):**

> *"Thank you, Angad. Slide 15 details our machine-checked invariants: Invariant G0 verifies every mentioned entity exists in the evidence store, G1 enforces bounded severity enums, G2 validates MITRE technique identifiers, and G3 mandates active provenance pointers."*

- **Slide 16 (Empirical Evaluation & Future Scope):**

> *"AACIP achieved a 100% pass rate across 244 automated tests, with a 0.0% entity hallucination rate across all benchmark modalities. Our future scope includes short-term integration with dynamic sandboxes like CAPE, and long-term enterprise SIEM connectors for distributed SOC deployments."*

- **Slide 17 (Conclusion):**

> *"In conclusion, AACIP proves that generative AI can be safely deployed in digital forensics when constrained by formal grounding invariants, delivering full auditability and significantly reducing analyst triage time."*

- **Slides 18 & 19 (References & Q&A):**

> *"Slide 18 lists our foundational literature and standards. We thank our guide Dr. Vanitha Soman and the examination panel for their time. We are now open for questions."*

---

3. Technical Preparation Guide for Angad Mistry

As presenter of **Slides 9 through 14**, master these core concepts and defensive responses:

Core Concepts to Review

1. **Finite State Machine Lifecycle:**
   `INITIALIZED` ➔ `INGESTING` ➔ `EXTRACTING` ➔ `CORRELATING` ➔ `REASONING` ➔ `VALIDATING` ➔ `COMPLETED`.
   Implementation: *`triage.py`* & *`investigation.py`*.
2. **Evidence DAG Construction:**
   Extracted entities become typed Pydantic nodes in the database. The relationship between parent entities (e.g., source IP) and child events (e.g., failed SSH login) forms a Directed Acyclic Graph (DAG) with causal edges.
3. **Dynamic SVG Rendering:**
   The web frontend (`web/app.js`) fetches the evidence adjacency list as JSON and dynamically generates scalable vector graphics (SVG) with coordinates and event handlers, avoiding bloated frontend dependencies.
4. **Model Abstraction & Offline Replay:**
   `core/llm/providers/reasoning.py` wraps OpenAI-compatible endpoints (NVIDIA NIM, DeepSeek-R1, Nemotron-70B). The platform includes a local Replay Provider for offline deterministic execution.

High-Yield Examiner Questions for Angad Mistry

- **Q: "Why use a multi-agent pipeline instead of a single prompt with few-shot examples?"**
- **Answer:** *"Few-shot prompts do not provide hard computational boundaries. Log files can contain adversarial prompt injection strings. In AACIP, deterministic tools parse the data first into isolated database entities; the LLM is only passed structured IDs, neutralizing prompt injection and context window bloat."*
- **Q: "How does the system ensure the LLM doesn't output ungrounded data?"**
- **Answer:** *"The Invariant Validator inspects the JSON payload returned by the LLM before writing to the database. If an entity or technique ID cannot be resolved against existing evidence keys (Invariant G0), the finding is rejected."*
- **Q: "How is real-time status delivered to the analyst interface?"**

- **Answer:** "*FastAPI handles asynchronous request orchestration. As each stage in the state machine transitions, status updates and logs are dispatched to the client via WebSockets and REST polling.*"